MOODIFY – Mood Based Music Playlist & Streaming Application

Project Title: MOODIFY – Mood Based Music Playlist & Streaming Application
Team Name:
Team Members: **Chaturvi P**
Aniket Menon
Abhiram A
Adithiyaa Kala Kandan
Aryav Agarwal
Ayush S Kulkarni

College Name: RV University
Department: CSE
Academic Year: 2025–2026

Abstract

Moodify is a mobile music application designed to provide users with a clean, engaging, and personalized music listening experience. The project focuses on combining modern Android user interface design with essential music management features such as authentication, playlist handling, favorites, recently played tracking, profile management, and player interaction. The application is developed as an academic mini-project to demonstrate practical knowledge of Android development, user-centered interface design, and backend integration.

The main purpose of Moodify is to address the inconvenience users often face while navigating ordinary playlist systems that are not aligned with mood, convenience, or personalization. In many basic music applications, users must manually organize songs, search repeatedly and manage disconnected playlists. Moodify aims to simplify this by creating a more intuitive flow in which users can quickly access songs, maintain playlists, mark favorites, and revisit recently played content.

An important idea behind the project is the connection between music and human emotion. Users often prefer music according to mood, activity, or atmosphere. A mood-oriented music application improves the overall user experience by making music selection feel more natural and personal. Even in its current prototype stage, Moodify reflects this concept through structured content organization, visually expressive design, and a user-friendly navigation model.

The application is developed using Kotlin and XML in Android Studio. Firebase Authentication is used for user login and signup functionality, while SharedPreferences is used to manage session persistence and selected local user data. RecyclerView components are used for dynamic content presentation, and the modular screen-based design helps organize the application into manageable and scalable functional units.

Overall, Moodify serves as a strong foundation for a future-ready music streaming platform. It demonstrates essential engineering concepts including application architecture, UI/UX planning, authentication flow, reusable components, and local data handling while also presenting a polished interface suitable for academic evaluation.

Introduction

Music streaming applications have become one of the most frequently used categories of mobile software. With the increasing use of smartphones and on-demand digital entertainment, users expect fast, beautiful, and personalized access to music. Modern applications such as Spotify, Apple Music, and YouTube Music have influenced user expectations by offering easy navigation, recommendations, playlists, and immersive music player experiences.

Despite this progress, many simplified or basic music applications still face common limitations. Traditional playlist systems often require too much manual effort. Users may need to browse through multiple screens, search repeatedly for songs, and manage playlists without any emotional or contextual support. In many cases, the interface is functionally correct but lacks a smooth and enjoyable user experience.

Music is deeply connected to emotion, lifestyle, and personal routine. Users do not always choose songs by title or artist alone; they often choose songs according to how they feel. This creates the need for mood-based and personalized systems that can make music interaction more natural. A system that supports mood-based access, favorites, recently played tracking, and personalized playlists can improve both satisfaction and engagement.

Moodify is developed with this need in mind. The project is designed as an Android-based music playlist and streaming UI application that combines essential features with modern interface design. It provides authentication, smooth page transitions, user profile handling, favorites, playlist creation, and a music player interface. The project highlights both technical implementation and design thinking, making it suitable as an engineering mini-project.

Problem Statement

Many users experience difficulty when managing music in applications that are not designed with personalization and convenience in mind. Standard playlist systems often feel static, requiring repeated manual effort to organize songs, switch between playlists, and locate preferred content. This reduces efficiency and negatively affects the listening experience.

Another major issue is the lack of emotional personalization. Music preference often changes depending on a user's mood, time of day, or activity. However, many systems do not provide a user experience that reflects this behavior in a simple and accessible way. As a result, users may spend more time searching for appropriate songs than actually enjoying them.

Users also expect visually smooth and intuitive applications. Poor navigation, cluttered interfaces, and inconsistent layouts create friction. Therefore, there is a clear need for an application that provides a modern, simple, and user-friendly music experience while supporting playlist management, favorites, listening history, and scalable personalization.

Objectives

The major objectives of the Moodify project are listed below:

- To design and develop a modern Android-based music application with an attractive user interface.
- To provide a personalized and mood-oriented music interaction model.
- To implement secure user login and signup functionality.
- To enable users to create, view, and manage playlists easily.
- To provide a favorites system for saving preferred songs.
- To maintain recently played song history for quick revisit.

- To implement smooth navigation across all screens of the application.
- To demonstrate the use of Android components such as Activities, RecyclerView, Adapters, Intents, and SharedPreferences.
- To integrate Firebase services for authentication and future database expansion.
- To create a strong prototype that can be extended into a real-world music platform.

Technology Stack

Core Technologies

Layer	Technology Used	Purpose
Frontend	Kotlin	Application logic and Android development
UI Design	XML Layouts	Screen design and interface structuring
Design System	Material Design principles	Modern and consistent UI behavior
Backend Service	Firebase Authentication	User login, signup, and session handling
Local Storage	SharedPreferences	Store login state, user preferences, and lightweight local data
Dynamic Lists	RecyclerView	Efficient display of music items and recent items
IDE	Android Studio	Development, debugging, and UI design

Why These Technologies Were Selected

Kotlin was selected because it is the officially recommended language for Android development. It provides concise syntax, strong null safety, and easier maintenance when compared to older Android development approaches.

XML layouts were used because they provide clear separation between user interface structure and application logic. This makes screens easier to design, edit, and reuse.

Firebase Authentication was chosen because it simplifies secure user management and reduces the effort required to build authentication from scratch. It is reliable, scalable, and suitable for student mini-project implementation.

SharedPreferences was selected for lightweight local persistence. It is useful for storing session-related values, small settings, and simple user-side information without requiring a full local database.

RecyclerView was used because music applications typically display repeated lists such as songs, recent tracks, and playlists. RecyclerView improves performance and supports flexible custom layouts.

Android Studio was used as the integrated development environment because it provides complete Android SDK support, emulator tools, layout preview, Gradle integration, and debugging assistance.

Additional Technical Notes

- Authentication is implemented using Firebase user credentials.
- Firestore or another cloud database can be integrated in later versions for playlist and song persistence.
- State management is handled through Activity flow, Intents, and local persistence rather than a complex architecture framework.
- The current project is suitable for prototype demonstration and future modular extension.

System Architecture

Moodify follows a simple Activity-based Android architecture in which each major function is represented by a dedicated screen. This architecture is appropriate for a mini-project because it is straightforward, easy to understand, and suitable for demonstrating core Android concepts.

Overall Flow

The general workflow of the application can be described as follows:

1. The user launches the application.
2. Splash screen checks whether the user is already authenticated.
3. If authenticated, the user is redirected to the Home Page.
4. If not authenticated, the user is taken to Login or Signup.
5. After login, the user can browse music sections, open playlists, mark favorites, access profile information, and navigate to the player screen.

Functional Architecture

```
Splash Screen
  ↓
Login / Signup
  ↓
Home Page
 ├── Profile Page
 │    └── Profile Editor
 ├── Music Player
 ├── Favourites
 ├── Playlist List
 │    ├── Playlist Page
 │    └── Create New Playlist Page
 └── Recently Played
```

Data Flow Explanation

- User credentials are submitted from the Login or Signup page.
- Firebase Authentication validates the credentials.
- On successful authentication, the session is maintained for later access.
- Music item data is displayed through RecyclerView and Adapter components.
- Favourites and recently played states can be stored locally using SharedPreferences.
- Playlist-related data is currently represented as UI flow and can later be connected to Firestore.

Authentication Flow

The authentication module checks whether a valid user session exists. If the session exists, the app bypasses the login page and improves user convenience. If not, the user must authenticate before gaining access to protected pages.

Playlist Management Workflow

The playlist management workflow allows users to access a list of playlists, open an individual playlist, and create a new playlist. In the current implementation, this reflects the core user flow and UI structure, while future versions can persist this data using cloud storage.

Music Playback Workflow

When a user selects a song card or recent item, the app navigates to the Music Player screen. The player presents the selected song details, playback controls, and a seek bar. This workflow is designed to simulate real playback behavior and provide a polished streaming app experience.

Module / Page Explanation

1. Login Page

Purpose:
The Login Page is used to authenticate existing users and provide secure access to the application.

Features:

- Email input field
- Password input field
- Login button
- Navigation to Signup page
- Form validation and authentication support

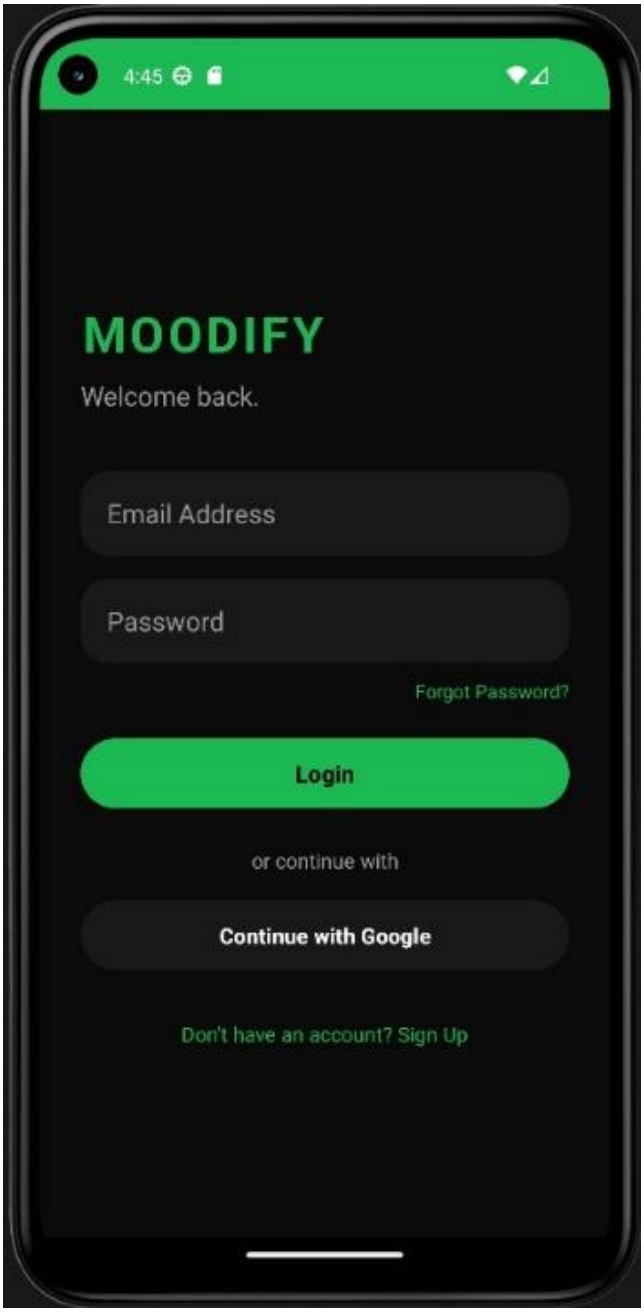
UI Explanation:
The page follows a clean card-based layout with input fields and a prominent action button. The design is intended to reduce visual clutter and help users complete authentication quickly.

Functionalities:

- Accepts user credentials
- Validates input values
- Sends login request to Firebase Authentication
- Redirects to Home Page after successful login

User Interaction Flow:
The user opens the app, enters email and password, taps the login button, and receives access upon successful validation.

Image Placeholder:



2. Home Page

Purpose:
The Home Page acts as the main dashboard of the application and gives users access to major content sections and navigation points.

Features:

- Hero banner or featured section
- Music categories or top mix section
- Recently played items
- Profile access
- Navigation to player and playlists

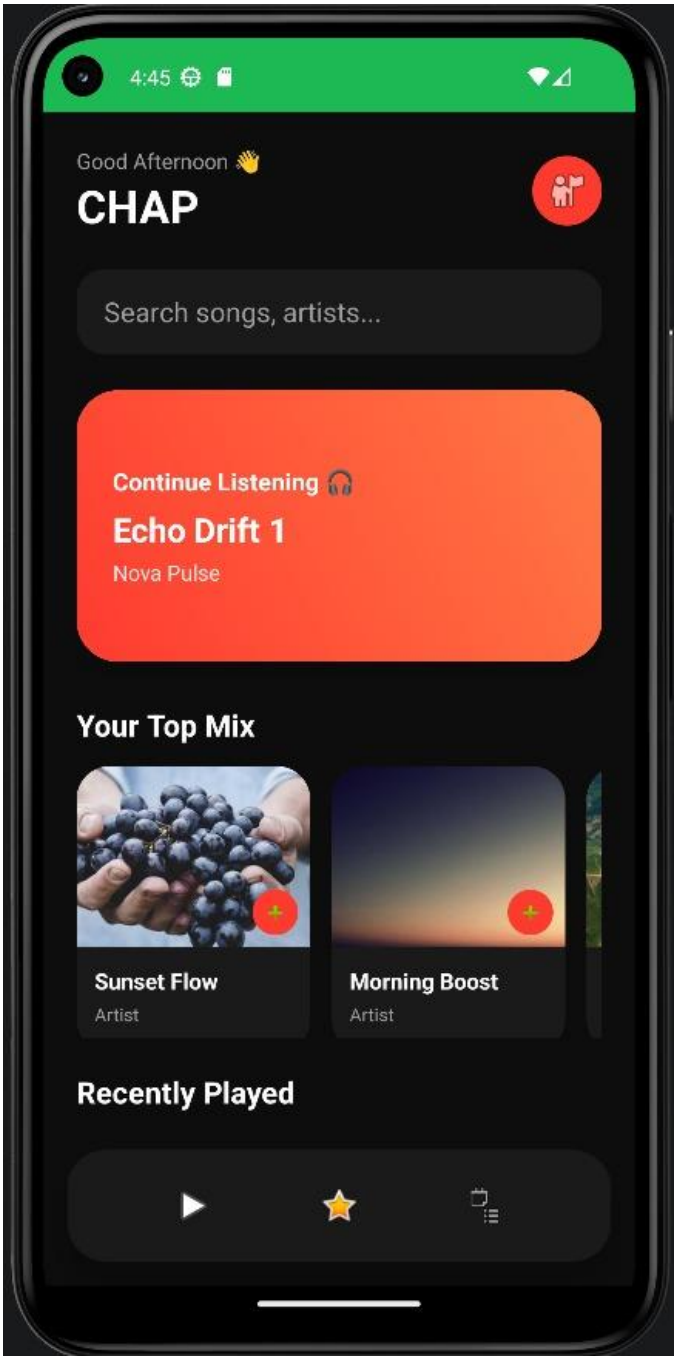
UI Explanation:
The Home Page is designed to be visually attractive and easy to scan. It combines banners, cards, and RecyclerView sections to create a modern streaming interface.

Functionalities:

- Displays song collections and recent items
- Provides entry points to music playback and profile management
- Supports smooth scrolling and quick interaction

User Interaction Flow:
After login, the user reaches the Home Page, explores music cards, taps desired content, and navigates to the relevant section.

Image Placeholder:



3. Profile Page

Purpose:
The Profile Page presents user-related information and account-level actions.

Features:

- Display user details
- Access profile editing
- Logout option
- Placeholder for settings or preferences

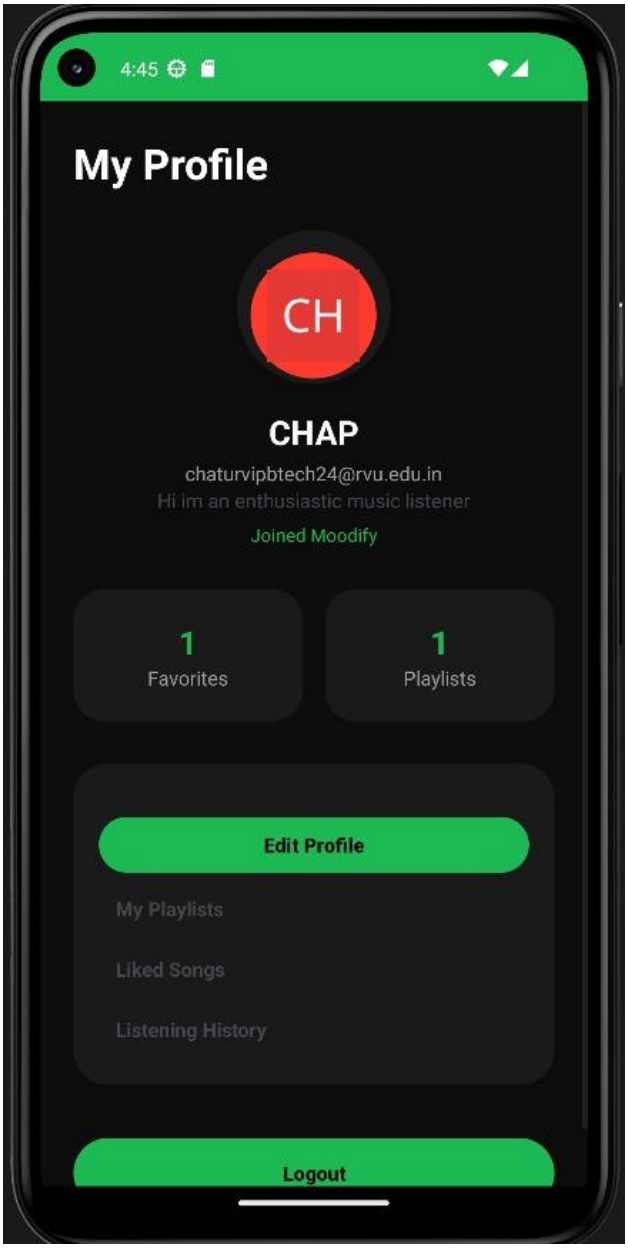
UI Explanation:
The interface is minimal and organized to keep account actions clear and separate from music browsing functions.

Functionalities:

- Shows current user information
- Allows access to editing screen
- Supports logout and session control

User Interaction Flow:
The user opens the profile from the home screen, views account information, and can either edit details or log out.

Image Placeholder:



4. Profile Editor

Purpose:
The Profile Editor page allows the user to update personal details and improve customization.

Features:

- Editable profile fields
- Save or update action
- Cleaner personalization support

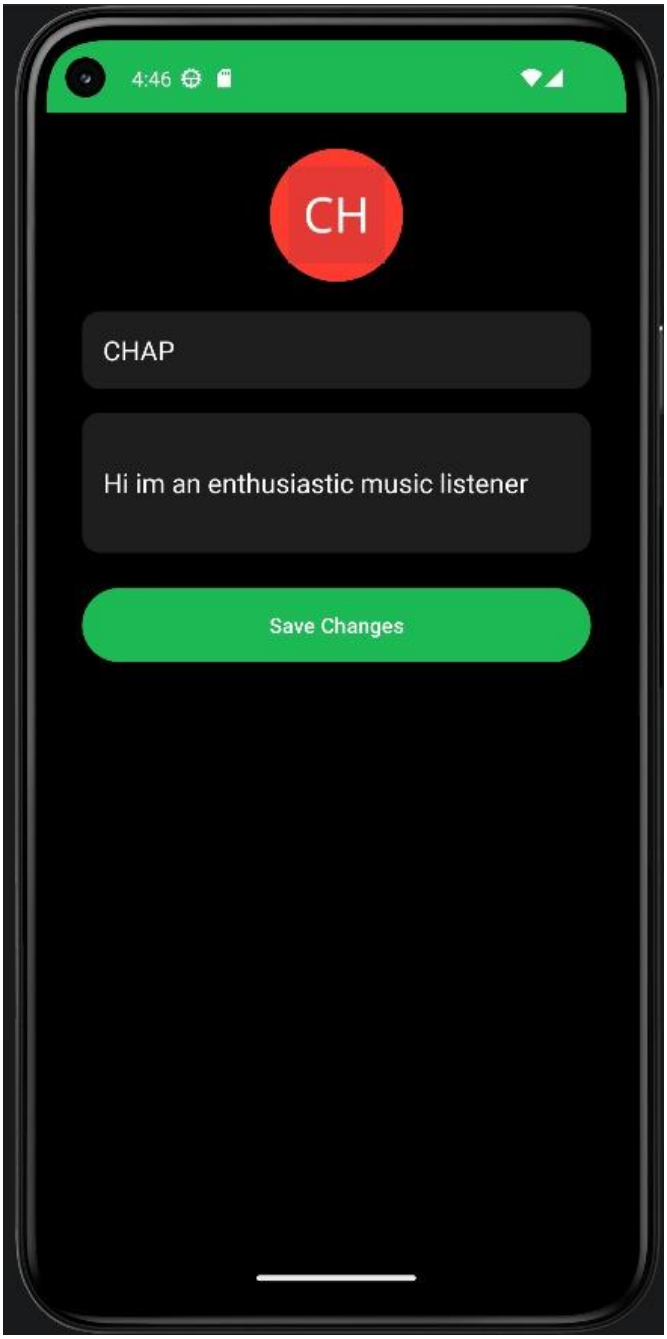
UI Explanation:
The screen is form-based and designed to remain simple so the user can modify profile data without confusion.

Functionalities:

- Accepts editable user inputs
- Stores updated values locally or through future database support
- Improves personalization capability

User Interaction Flow:
The user moves from Profile Page to Profile Editor, updates fields, and saves changes.

Image Placeholder:



5. Music Player

Purpose:

The Music Player is the core interaction screen for playing and controlling selected songs.

Features:

- Album artwork display

- Song title and artist details
- Play/pause controls
- Seek bar
- Previous/next style controls

UI Explanation:
The player is designed to feel immersive, with larger visual emphasis on album artwork and centralized controls. This improves focus and gives a premium app feel.

Functionalities:

- Displays selected song information
- Simulates or controls playback interaction
- Supports navigation back to previous screens

User Interaction Flow:
The user selects a song from the Home Page, playlist, or recent list and is redirected to the player to control playback.

Image Placeholder:

6. Favourites

Purpose:

The Favourites page allows users to maintain a list of preferred songs for quick access.

Features:

- Display saved favourite songs

- Add/remove favorite option
- Quick revisit to liked tracks

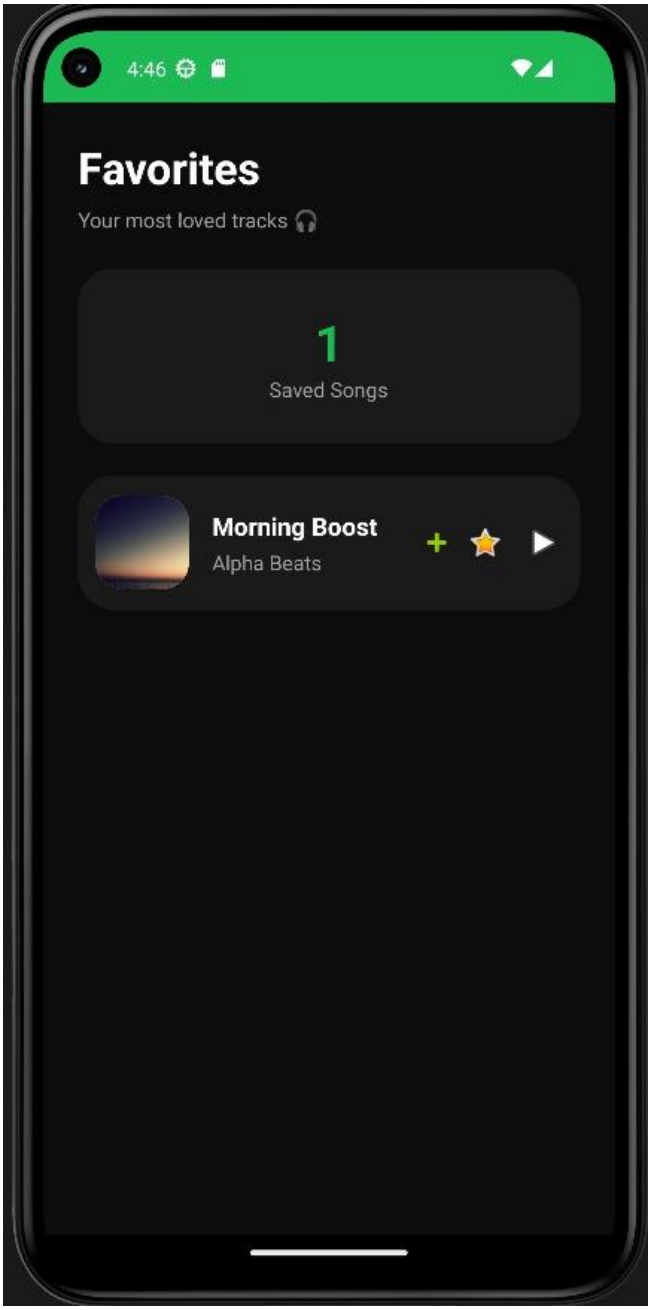
UI Explanation:
The layout is list-oriented and designed for convenience. Users should be able to identify and open favourite tracks with minimal effort.

Functionalities:

- Stores and retrieves favourite selections
- Updates list dynamically based on user action
- Integrates with local storage logic

User Interaction Flow:
The user marks songs as favorite while browsing or playing music and later accesses all liked songs from the Favourites page.

Image Placeholder:



7. Playlist List

Purpose:
This page displays all user playlists and acts as the main playlist management entry point.

Features:

- Playlist overview
- Navigation to individual playlist page
- Access to create playlist option

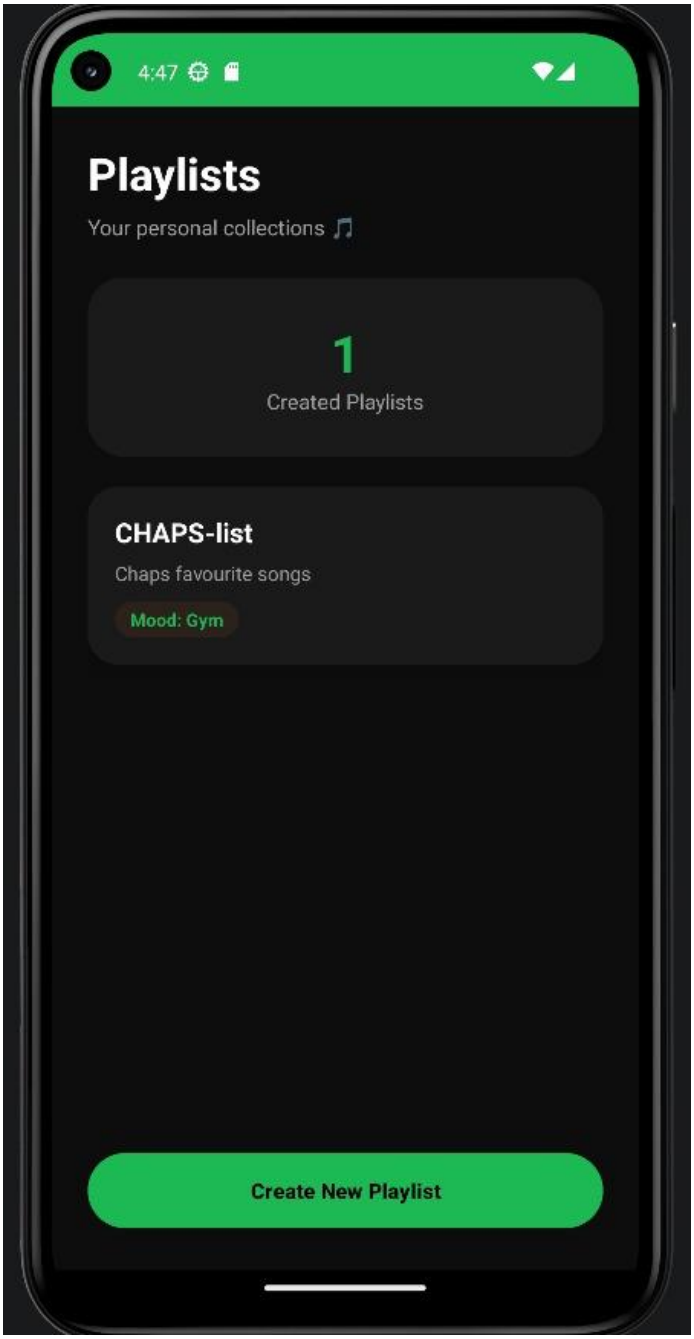
UI Explanation:
The interface uses card or list patterns to clearly display available playlists and encourage organized browsing.

Functionalities:

- Shows available playlists
- Supports selection of a specific playlist
- Links to new playlist creation

User Interaction Flow:
The user opens the playlist section, views available playlists, and selects one to access detailed contents.

Image Placeholder:



8. Playlist Page

Purpose:

The Playlist Page shows the songs contained within a selected playlist.

Features:

- Playlist title display
- Song list within selected playlist
- Navigation to music player
- Playlist content view

UI Explanation:

The layout is content-focused, allowing the user to concentrate on the selected playlist and its songs.

Functionalities:

- Displays songs linked to a playlist
- Allows opening any song in the player
- Prepares structure for future cloud sync support

User Interaction Flow:

The user selects a playlist from Playlist List, views all included songs, and chooses a track to play.

Image Placeholder:

9. Create Playlist Page

Purpose:
The Create Playlist Page allows users to generate custom playlists according to their mood, genre preference, or activity.

Features:

- Playlist name input
- Create button
- Future song selection support

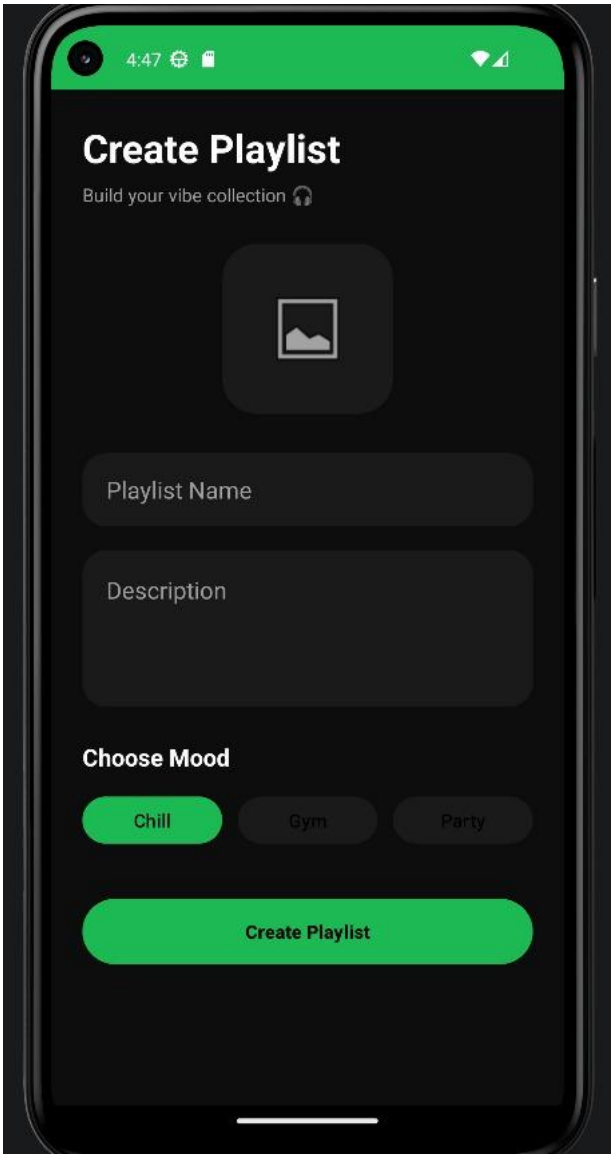
UI Explanation:
The screen is intentionally simple, keeping the focus on creation rather than navigation complexity.

Functionalities:

- Accepts playlist title or related details
- Creates a new playlist entry
- Can later be integrated with database persistence

User Interaction Flow:
The user chooses to create a new playlist, enters a name, confirms the action, and returns to the playlist collection.

Image Placeholder:



10. Recently Played

Purpose:

The Recently Played section helps users quickly revisit songs they have listened to earlier.

Features:

- History of recently accessed tracks
- Fast reopening of songs
- Supports user convenience and continuity

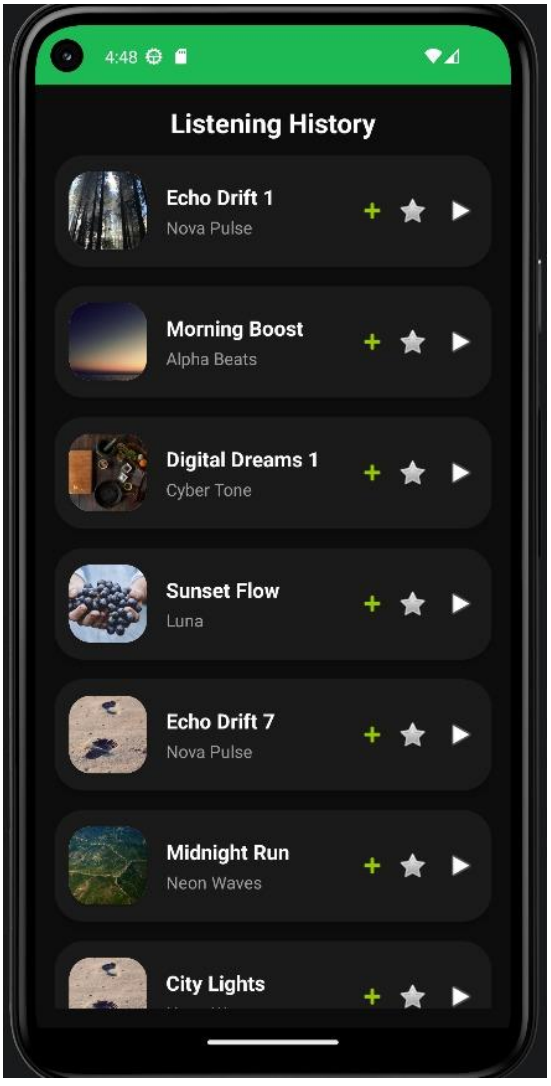
UI Explanation:
The section is compact and typically integrated into the Home Page or a dedicated list view. It is designed for quick recognition and one-tap access.

Functionalities:

- Tracks previously opened songs
- Displays them in order of recent interaction
- Improves repeat listening experience

User Interaction Flow:
As the user listens to songs, the app stores those interactions and presents them in the Recently Played section for easy return.

Image Placeholder:



UI/UX Design Analysis

The user interface of Moodify is designed to create a premium and modern music experience while remaining simple enough for quick interaction. A professionally styled interface is important in music applications because users expect both visual appeal and usability.

Theme and Visual Identity

The application can be presented effectively using a dark theme with green accent highlights, as this style is strongly associated with modern music applications. A dark background reduces visual fatigue and creates a cinematic atmosphere for album artwork, cards, and playback controls. Green accents can be used for active buttons, selected states, and highlighted actions, helping important elements stand out clearly.

Minimalistic Design

A minimal interface improves usability by avoiding unnecessary visual clutter. In Moodify, major actions such as play, pause, favorite, profile access, and playlist navigation should remain immediately visible or easy to locate. This supports a smooth and low-friction user journey.

Button Placement and Navigation

Buttons should be placed according to user priority. Login and signup actions should be highly visible. Playback controls should remain centrally positioned for faster interaction. Profile, playlist, and favorites access should be consistent across screens so users do not need to relearn navigation.

Accessibility Considerations

Readable text contrast, touch-friendly buttons, and clear spacing between UI elements improve accessibility. The app should maintain sufficient font size and spacing so users can comfortably interact with the interface on a range of screen sizes.

Design Psychology

Music applications benefit from mood-oriented visual design. Dark themes, vibrant accents, rounded cards, and immersive artwork help create an emotional connection between the interface and the listening experience. This design language makes the app feel more engaging and aligned with the concept of mood-based music consumption.

Database Design

The current implementation uses Firebase Authentication and local storage concepts, while future versions can use Firebase Firestore for full data persistence. The database design can be described using the following entities.

Main Data Entities

Entity	Description	Example Fields
User	Stores user-related information	userId, name, email, profileImage
Playlist	Stores playlist metadata	playlistId, userId, playlistName, createdAt
PlaylistSongs	Maps songs to playlists	playlistId, songId
Favourites	Stores liked songs for each user	userId, songId
RecentlyPlayed	Maintains playback history	userId, songId, playedAt
Songs	Stores song metadata	songId, title, artist, album, imageUrl, moodTag

Authentication Records

User authentication details are handled through Firebase Authentication. This ensures secure login, signup, and session persistence without requiring manual password storage in the app database.

Playlist Storage

Each playlist can be linked to a specific user. A single user can create multiple playlists, and each playlist can contain multiple songs.

Favourite Songs

Favourite songs are associated with user identity so that each user maintains a separate personalized list.

Recently Played Songs

This data helps improve usability by allowing users to revisit songs they played earlier. It can be stored locally or synchronized to a cloud database in future updates.

Music Metadata

Song-level data includes display information such as title, subtitle, artist name, album artwork, and mood or genre category. This structure supports future search, filtering, and recommendation features.

Features of the Application

Moodify includes the following major application features:

- **User Authentication:** Secure signup, login, session handling, and logout.
- **Home Dashboard:** Central screen for discovering songs and navigating the application.
- **Mood-Oriented Experience:** Structured presentation that supports emotion-based listening behavior.
- **Playlist Creation:** Users can create and manage custom playlists.
- **Favourites System:** Songs can be marked and revisited quickly.
- **Recently Played Tracking:** The application maintains recent listening history.
- **Music Player Interface:** Interactive player with artwork, controls, and seek bar.
- **Profile Management:** User account page and editable profile details.
- **Dynamic UI Rendering:** RecyclerView-based presentation for songs and lists.
- **Responsive Layout:** Layouts are designed for mobile usability and clean visual hierarchy.

Innovative Features

Although Moodify is a mini-project prototype, it includes several innovative and user-centric ideas:

- **Mood-Based Categorization Concept:** The project is built around the idea that music selection is strongly connected to emotion and atmosphere.
- **Personalized Interaction:** Features like favorites, recently played, and profile management contribute to a customized user experience.
- **Emotion-Driven Playlist Thinking:** The concept of user-created playlists can be aligned with mood, productivity, relaxation, or activity.

- **Modern Interface Language:** Rounded cards, immersive player layout, and premium app styling improve perceived quality.
- **Scalable Foundation:** The project can evolve into a fully intelligent music platform with recommendation and streaming support.

Challenges Faced

During the development of a project like Moodify, several practical engineering challenges may arise:

- Maintaining UI consistency across multiple screens.
- Designing layouts that remain visually attractive without becoming cluttered.
- Managing activity navigation and back-stack behavior correctly.
- Synchronizing music selection flow with the player screen.
- Handling local persistence for favorites and recently played songs.
- Integrating Firebase Authentication smoothly with form validation.
- Preparing the application structure for future database integration.
- Ensuring that playlist handling remains simple and understandable for the user.

These challenges are common in Android development and provide valuable learning outcomes in both design and implementation.

Testing

Testing is an essential part of validating application quality. The Moodify project can be tested using the following approaches:

Functional Testing

This ensures each major feature works according to requirements. Login, signup, navigation, playlist creation flow, favorite marking, and player access should all be tested individually.

UI Testing

User interface testing verifies alignment, spacing, responsiveness, readability, and interaction clarity. Every screen should be checked for layout consistency and usability on different device sizes.

Navigation Testing

This ensures that screen transitions occur correctly and that buttons open their intended destinations. Back navigation should also behave properly.

Authentication Testing

Authentication testing checks valid login, invalid login, account creation, session persistence, and logout functionality.

Playlist and Favourites Testing

Playlist creation logic and favourites toggling should be tested to ensure user actions are correctly reflected in the interface and stored data.

Bug Fixing

Bugs discovered during testing should be documented, reproduced, and corrected. Common examples include null input issues, incorrect intent targets, or state persistence errors.

Future Enhancements

Moodify has strong potential for future expansion. The following improvements can be added in advanced versions:

- AI-based music recommendation according to listening behavior.
- Smart mood detection using user input, activity pattern, or time context.
- Real audio playback engine for actual song streaming.
- Spotify or third-party music API integration.
- Offline music access and download support.
- Cloud synchronization for playlists and favorites.
- Profile photo upload and richer user personalization.
- Dark/light theme switching.

- Social sharing of playlists.
- Voice assistant or voice-based search support.
- Real-time updates across multiple devices.

Conclusion

Moodify is a well-structured Android mini-project that combines music application design with practical engineering implementation. The project successfully demonstrates important areas of Android development including authentication, user interface design, activity navigation, dynamic list rendering, local data persistence, and modular page organization.

The application provides a polished and user-friendly experience through features such as playlists, favorites, recently played tracking, profile management, and an interactive music player screen. Its concept of mood-based music interaction adds originality and makes the project more engaging from both a technical and user-experience perspective.

From an academic point of view, the project reflects meaningful learning in software design, UI/UX planning, Firebase integration, and Android application development. It also provides a scalable foundation that can be extended into a more advanced music recommendation and streaming platform in future work.

References

1. Android Developers Documentation – Kotlin for Android
2. Android Developers Documentation – RecyclerView
3. Firebase Documentation – Firebase Authentication
4. Material Design Guidelines
5. Android Studio Official Documentation
6. UI inspiration from modern music streaming applications such as Spotify and Apple Music
7. General mobile UI/UX design references for card-based layouts, dark theme interfaces, and media player interaction design

- d component.

- Describe playlists, favourites, and recent tracking as personalization features.
- Keep the demo smooth and sequential.
- End with future scope and learning outcomes.